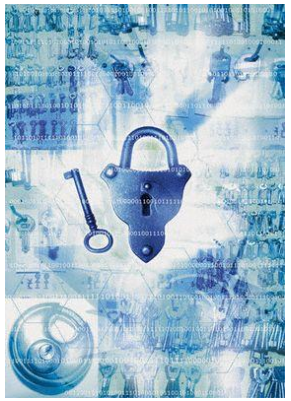
Information Security



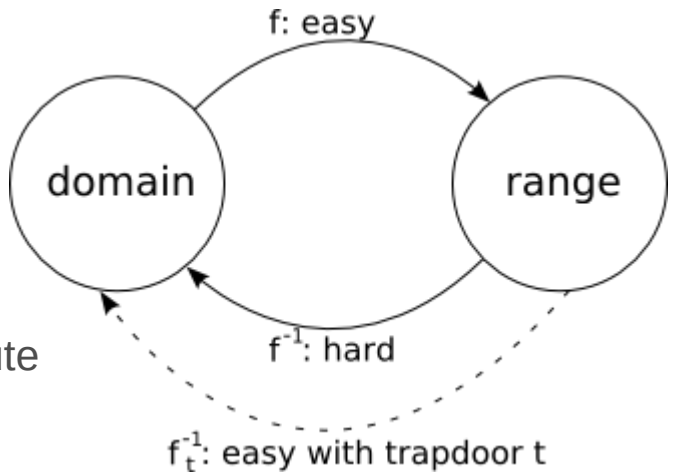
Public Key Crypto

AI컴퓨터공학부
김 희 열

Public Key Cryptography

- Two keys
  - Sender uses recipient's **public key** to encrypt
  - Receiver uses his **private key** to decrypt

- Based on **trap door, one way function**
  - Easy to compute in one direction
  - Hard to compute in other direction
  - "Trap door" used to create keys
  - Example
    - Given p and q, product N=pq is easy to compute
    - But given N, it is hard to find p and q



- $f(13,17) = 13 \times 17 = 221$
- $221 = ? \times ?$ ← difficult?
  - Yes, if N is very large

Public Key Cryptography

- Encryption
  - Suppose we encrypt M with Bob's public key
  - Only Bob's private key can decrypt to find M

- Digital Signature
  - **Sign** by "encrypting" with private key
  - Anyone can **verify** signature by "decrypting" with public key
  - But only private key holder could have signed
  - Like a handwritten signature (and then some)

Knapsack Problem

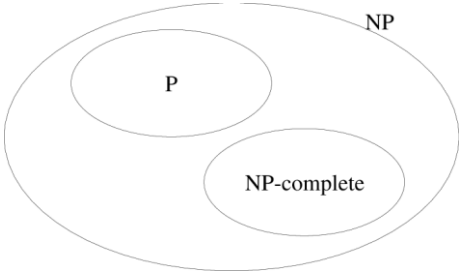
- Given a set of n weights $W_0, W_1, \ldots, W_{n-1}$ and a sum S, is it possible to find $a_i \in \{0,1\}$ so that

  $$S = a_0 W_0 + a_1 W_1 + \ldots + a_{n-1} W_{n-1}$$

  (technically, this is "subset sum" problem)

- **Example**
  - Weights $(62,93,26,52,166,48,91,141)$
  - Problem: Find subset that sums to $S=302$
  - Answer: $62+26+166+48=302$

- The (general) knapsack is NP-complete



Knapsack Problem

- General knapsack (GK) is hard to solve
- But **superincreasing knapsack** (SIK) is easy
  - each weight greater than the sum of all previous weights

- **Example**
  - Weights $(2,3,7,14,30,57,120,251)$
  - Problem: Find subset that sums to $S=186$
  - Work from largest to smallest weight
  - Answer: $120+57+7+2=186$

Knapsack Cryptosystem

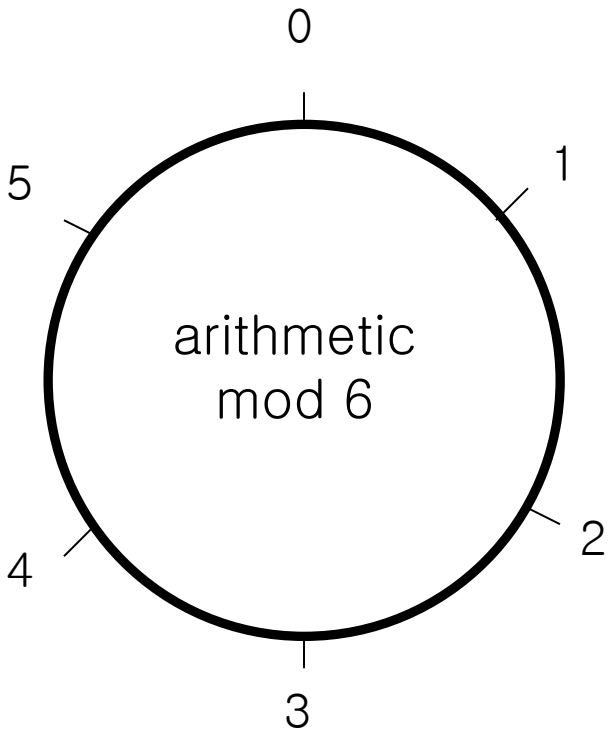
1. Generate superincreasing knapsack (SIK)
2. Convert SIK into “general” knapsack (GK)
3. **Public Key:** GK
4. **Private Key:** SIK + conversion factors

- Easy to encrypt with GK
- With private key, easy to decrypt
  - convert ciphertext to SIK
- Without private key, must solve GK (how??)

"Clock" Arithmetic

- For integers x and n, x mod n is the remainder of x ÷ n

- Examples
  - 7 mod 6 = 1
  - 33 mod 5 = 3
  - 33 mod 6 = 3
  - 51 mod 17 = 0
  - 17 mod 6 = 5



Modular Addition

- Notation and facts
  - 7 mod 6 = 1
  - 7 = 13 = 1 mod 6
  - ((a mod n) + (b mod n)) mod n = (a + b) mod n
  - ((a mod n)(b mod n)) mod n = ab mod n

- Addition Examples
  - 3 + 5 = 2 mod 6
  - 2 + 4 = 0 mod 6
  - 3 + 3 = 0 mod 6
  - (7 + 12) mod 6 = 19 mod 6 = 1 mod 6
  - (7 + 12) mod 6 = (1 + 0) mod 6 = 1 mod 6

Modular Multiplication

- Multiplication Examples
  - $3 \cdot 4 = 0 \pmod 6$
  - $2 \cdot 4 = 2 \pmod 6$
  - $5 \cdot 5 = 1 \pmod 6$
  - $(7 \cdot 4) \bmod 6 = 28 \bmod 6 = 4 \bmod 6$
  - $(7 \cdot 4) \bmod 6 = (1 \cdot 4) \bmod 6 = 4 \bmod 6$

Modular Inverses

- Additive inverse of x mod n, denoted -x, is the number that must be added to x to get 0 mod n
  - -2 mod 6 = 4, since 2 + 4 = 0 mod 6

- Multiplicative inverse of x mod n, denoted x^{-1}, is the number that must be multiplied by x to get 1 mod n
  - 3^{-1} mod 7 = 5, since $3 \cdot 5 = 1$ mod 7

Modular Arithmetic Quiz

- Q: What is 3 mod 6?
- A: 3

- Q: What is -1 mod 6?
- A: 5

- Q: What is 5^{-1} mod 6?
- A: 5

- Q: What is 2^{-1} mod 6?
- A: No number works!

- Multiplicative inverse does not always exist

Relative Primality

- x and y are **relatively prime** if they have no common factor other than 1

- x^{-1} mod y exists only when x and y are relatively prime

- x^{-1} mod y is easy to find (when it exists) using the Extended Euclidean Algorithm
  - https://www.extendedeuclideanalgorithm.com/ 참조

Knapsack Example

- Alice generates a key pair
- Let (2,3,7,14,30,57,120,251) be the SIK
- Choose m = 41 and n = 491
  - m and n are relative prime
  - n is greater than sum of elements of SIK

- Convert to General Knapsack

$2 \cdot 41 \mod 491 = 82$
$3 \cdot 41 \mod 491 = 123$
$7 \cdot 41 \mod 491 = 287$
$14 \cdot 41 \mod 491 = 83$
$30 \cdot 41 \mod 491 = 248$
$57 \cdot 41 \mod 491 = 373$
$120 \cdot 41 \mod 491 = 10$
$251 \cdot 41 \mod 491 = 471$

➔ (82,123,287,83,248,373,10,471)

$w \cdot m \bmod n = w'$

Knapsack Example

- **Alice's Private key:** (2,3,7,14,30,57,120,251)

  $m^{-1} \bmod n = 41^{-1} \bmod 491 = 12$

- **Alice's Public key:** (82,123,287,83,248,373,10,471), n=491

Send S=548

convert to SIK quiz

S' = 548 · 12 = 193 mod 491

solve SIK

S' = 2 + 14 + 57 + 120

P = 10010110

P = 10010110

make a GK quiz

S = 82 + 83 + 373 + 10 = 548

$w \cdot m \bmod n = w'$

$w' \cdot m^{-1} \bmod n = w$

Knapsack Weakness

- **Trapdoor**
  - Convert SIK into “general” knapsack using modular arithmetic
- **One-way**
  - General knapsack is easy to encrypt, hard to solve
  - SIK is easy to solve

- This knapsack cryptosystem is **insecure**
  - Broken in 1983 with Apple II computer
  - The attack uses **lattice reduction**

- “General knapsack” is not general enough!
- This special knapsack is easy to solve!

RSA

- Invented by
  - Cocks (GCHQ)
  - Independently, by Rivest, Shamir and Adleman (MIT)

- Let p and q be two large prime numbers
- Let $N = pq$ be the **modulus**

- Choose e relatively prime to $(p-1)(q-1)$
- Find d such that $ed = 1 \bmod (p-1)(q-1)$
  - How?

- **Public key** is (N,e)
- **Private key** is d

RSA

- Encryption

Alice's private key : d

Alice's public key : N, e

$C = M^e \bmod N$

$M = C^d \bmod N$

message M

$C = M^e \bmod N$

RSA

- To encrypt message M compute
  - $C = M^e \bmod N$
- To decrypt C compute
  - $M = C^d \bmod N$
- Recall that e and N are public
- If attacker can factor N, he can use e to easily find d
  - Since $ed = 1 \bmod (p-1)(q-1)$
- Factoring the modulus breaks RSA
- But, it's very very hard!!!
- It is not known whether factoring is the only way to break RSA

Does RSA Really Work?

- Given $C = M^e \bmod N$ we must show
  - $M = C^d \bmod N = M^{ed} \bmod N$

- We'll use **Euler's Theorem**
  - If x is relatively prime to n then $\mathbf{x^{\varphi(n)} = 1 \bmod n}$

- Facts
  - $ed = 1 \bmod (p-1)(q-1)$
  - By definition of "mod", $ed = k(p-1)(q-1) + 1$
  - $\varphi(N) = (p-1)(q-1)$
  - Then $ed - 1 = k(p-1)(q-1) = k\varphi(N)$

- $M^{ed} = M^{(ed-1)+1} = M \cdot M^{ed-1} = M \cdot M^{k\varphi(N)}$
  $= M \cdot (M^{\varphi(N)})^k \bmod N = M \cdot 1^k \bmod N = M \bmod N$

Totient Function

- $\varphi(n)$ is the number of numbers (positive integers) less than n, relatively prime to n

- Examples
  - $\varphi(4) = 2$ since 4 is relatively prime to 3 and 1
  - $\varphi(5) = 4$ since 5 is relatively prime to 1,2,3 and 4
  - $\varphi(12) = 4$
  - $\varphi(p) = p-1$ if p is prime
  - $\varphi(pq) = (p-1)(q-1)$ if p and q prime

Simple RSA Example

- Example of RSA
  - Select “large” primes $p = 11, q = 3$
  - Then $N = pq = 33$ and $(p-1)(q-1) = 20$
  - Choose $e = 3$ (relatively prime to 20)
  - Find d such that $ed = 1 \bmod 20$, we find that $d = 7$ works

- **Public key:** $(N, e) = (33, 3)$
- **Private key:** $d = 7$

Simple RSA Example

- **Public key:** $(N, e) = (33, 3)$
- **Private key:** $d = 7$

- Suppose message $M = 8$

- Ciphertext C is computed as

$$C = M^e \bmod N$$
$$= 8^3 \bmod 33 = 512 \bmod 33$$
$$= 17 \bmod 33$$

- Decrypt C to recover the message M by

$$M = C^d \bmod N$$
$$= 17^7 \bmod 33$$
$$= 410{,}338{,}673 \bmod 33$$
$$= 12{,}434{,}505 * 33 + 8 \bmod 33$$
$$= 8 \bmod 33$$

More Efficient RSA

- Modular exponentiation example
  - $5^{20} = 95367431640625 = 25 \bmod 35$

- A better way: **repeated squaring**
  - $20 = 10100$ base 2
  - $(1, 10, 101, 1010, 10100) = (1, 2, 5, 10, 20)$
  - Note that $2 = 1 \cdot 2$, $5 = 2 \cdot 2 + 1$, $10 = 2 \cdot 5$, $20 = 2 \cdot 10$
  - $5^1 = 5 \bmod 35$
  - $5^2 = (5^1)^2 = 5^2 = 25 \bmod 35$
  - $5^5 = (5^2)^2 \cdot 5^1 = 25^2 \cdot 5 = 3125 = 10 \bmod 35$
  - $5^{10} = (5^5)^2 = 10^2 = 100 = 30 \bmod 35$
  - $5^{20} = (5^{10})^2 = 30^2 = 900 = 25 \bmod 35$

- No huge numbers and it's efficient!

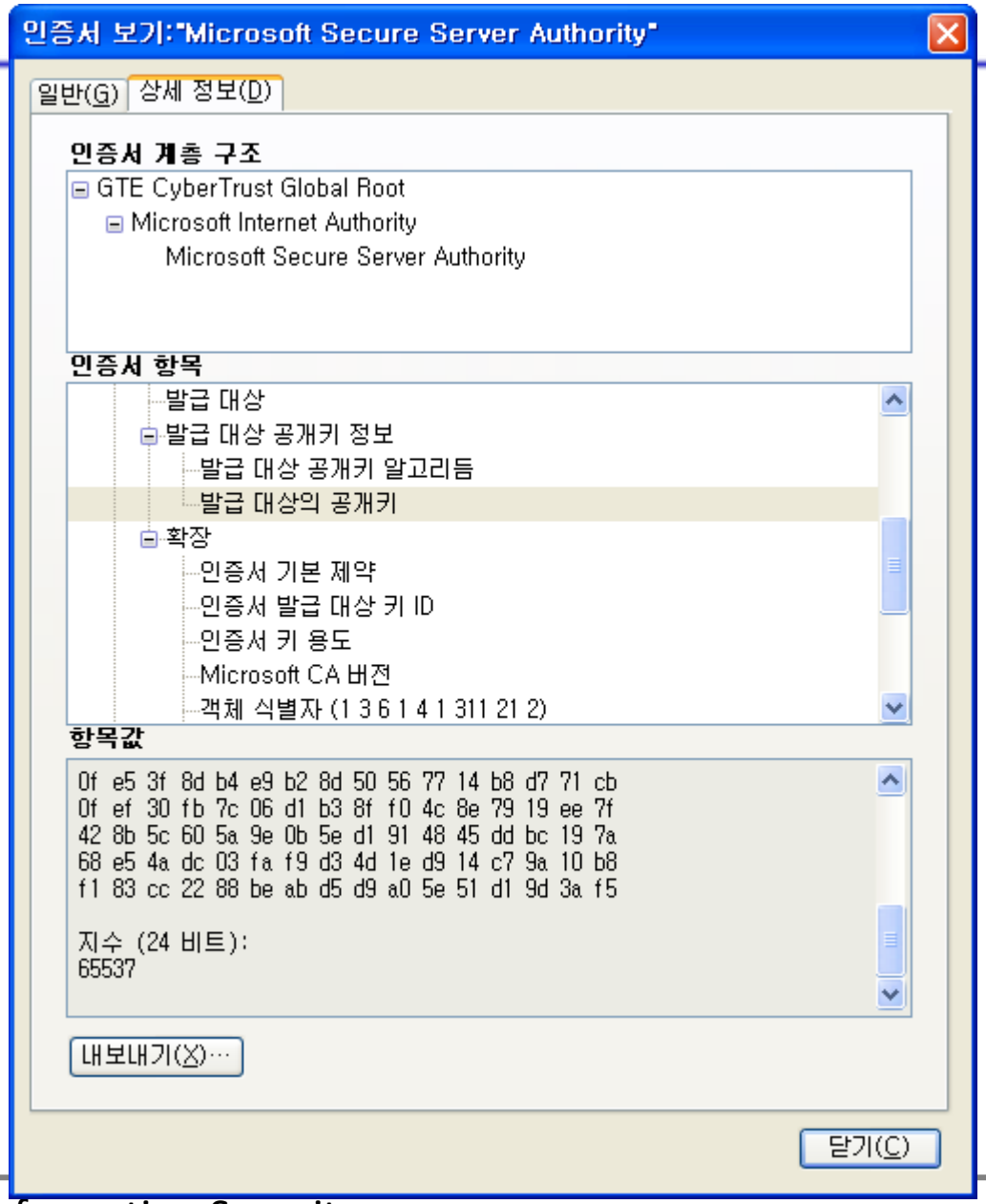
More Efficient RSA

- Let $e = 3$ for all users (but not same N or d)
  - Public key operations only require 2 multiplies
  - Private key operations remain “expensive”

  - If $M < N^{1/3}$ then $C = M^e = M^3$ and **cube root attack**
  - For any M, if C_1, C_2, C_3 sent to 3 users, cube root attack works
    - By using Chinese Remainder Theorem
  - Can prevent cube root attack by padding message with random bits

More Efficient RSA

- $e = 2^{16} + 1$ also used

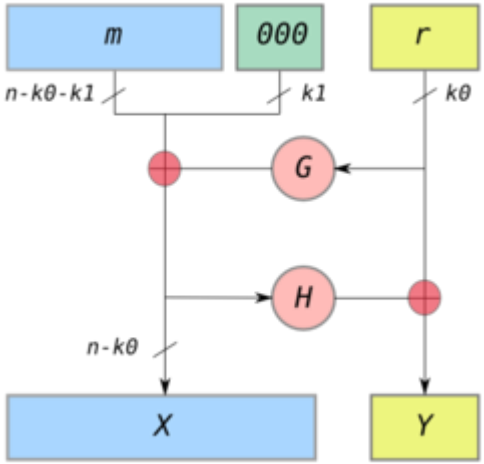


RSA padding

- There are attacks against plain RSA due to deterministic property
  - Solution: embed randomized padding before encryption

- Padding scheme
  - Optimal Asymmetric Encryption Padding (OAEP)

- When the industry refer to RSA, it is actually RSA with OAEP padding



HW

- Alice sent an RSA-encrypted message to Bob
  - Public key of Bob
    - n = 3174654383
    - e = 65537
  - Encrypted message
    - C = 2487688703
- Attack and get the plaintext & private key d

- Due : next class
  - Plaintext
  - d
  - Source code for factoring & decryption

- Tip
  - Using java BigInteger class is easy

Discrete Logarithm

- Fundamental to Diffie-Hellman key exchange & DSA
- Euler's Theorem
  - For every a and n that are relatively prime,

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

- If a and n are relatively prime, there is at least one integer m satisfies

$$a^{m} \equiv 1 \pmod{n}$$

  - The least positive integer m means
    - The order of a (mon n)
    - The length of the period generated by a
  - a is a primitive root (or generator) of n if m = φ(n)

$$\begin{aligned}
7^1 &\equiv && 7 \pmod{19} \\
7^2 &= 49 = 2 \times 19 + 11 && \equiv 11 \pmod{19} \\
7^3 &= 343 = 18 \times 19 + 1 && \equiv 1 \pmod{19} \\
7^4 &= 2401 = 126 \times 19 + 7 && \equiv 7 \pmod{19} \\
7^5 &= 16807 = 884 \times 19 + 11 && \equiv 11 \pmod{19}
\end{aligned}$$

Discrete Logarithm

- For a prime p, if a is a primitive root of p, then
  - a, a^2, …, a^{p-1} (mod p) are distinct

- primitive root of 19: 2, 3, 10, 13, 14, 15

Table 8.3 Powers of Integers, Modulo 19

a	a^2	a^3	a^4	a^5	a^6	a^7	a^8	a^9	a^{10}	a^{11}	a^{12}	a^{13}	a^{14}	a^{15}	a^{16}	a^{17}	a^{18}
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	4	8	16	13	7	14	9	18	17	15	11	3	6	12	5	10	1
3	9	8	5	15	7	2	6	18	16	10	11	14	4	12	17	13	1
4	16	7	9	17	11	6	5	1	4	16	7	9	17	11	6	5	1
5	6	11	17	9	7	16	4	1	5	6	11	17	9	7	16	4	1
6	17	7	4	5	11	9	16	1	6	17	7	4	5	11	9	16	1
7	11	1	7	11	1	7	11	1	7	11	1	7	11	1	7	11	1
8	7	18	11	12	1	8	7	18	11	12	1	8	7	18	11	12	1
9	5	7	6	16	11	4	17	1	9	5	7	6	16	11	4	17	1
10	5	12	6	3	11	15	17	18	9	14	7	13	16	8	4	2	1
11	7	1	11	7	1	11	7	1	11	7	1	11	7	1	11	7	1
12	11	18	7	8	1	12	11	18	7	8	1	12	11	18	7	8	1
13	17	12	4	14	11	10	16	18	6	2	7	15	5	8	9	3	1
14	6	8	17	10	7	3	4	18	5	13	11	2	9	12	16	15	1
15	16	12	9	2	11	13	5	18	4	3	7	10	17	8	6	14	1
16	9	11	5	4	7	17	6	1	16	9	11	5	4	7	17	6	1
17	4	11	16	6	7	5	9	1	17	4	11	16	6	7	5	9	1
18	1	18	1	18	1	18	1	18	1	18	1	18	1	18	1	18	1

Discrete Logarithm

- Ordinary logarithm
  - $y = x^i \leftrightarrow \log_x(y)=i$
  - $\log_x(1)=0$, $\log_x(x)=1$
  - $\log_x(yz)=\log_x(y)+\log_x(z)$
  - $\log_x(y^r)=r*\log_x(y)$

- Discrete logarithm
  - Consider a primitive root a for prime p
  - $a, a^2, \ldots, a^{p-1} \pmod p$ produce 1 ~ (p-1) exactly once
  - For any integer b, we can find a unique exponent i s.t. $b \equiv a^i \pmod p$ where $0 \le i \le p-1$
    - i : discrete logarithm of b for the base a (mod p)
    - $\text{dlog}_{a,p}(b) = i$
  - The difficulty seems to be the same order as that of factoring primes

(a) Discrete logarithms to the base 2, modulo 19

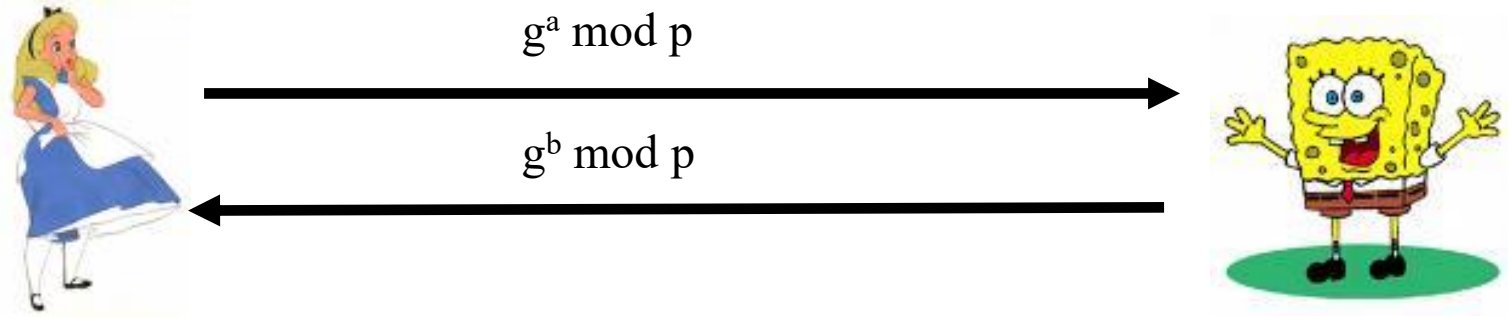
a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$log_{2,19}(a)$	18	1	13	2	16	14	6	3	8	17	12	15	5	7	11	4	10	9

Diffie-Hellman

- Invented by
  - Williamson (GCHQ)
  - Independently, by Diffie and Hellman (Stanford)
- A “key exchange” algorithm
  - Used to establish a shared symmetric key
- Not for encrypting or signing
- Security rests on difficulty of **discrete log** problem
  - given g, p, and $g^k \bmod p$ find k

Diffie-Hellman

- Let p be prime, let g be a **generator**
  - For any $x \in \{1,2,\ldots,p-1\}$ there is n s.t. $x = g^n \bmod p$
- Alice randomly selects secret value $a \in \{1, \ldots, p-1\}$
- Bob randomly selects secret value $b \in \{1, \ldots, p-1\}$



generate a
$(g^b)^a \bmod p = g^{ab} \bmod p$

generate b
$(g^a)^b \bmod p = g^{ab} \bmod p$

- Could use $K = g^{ab} \bmod p$ as symmetric key

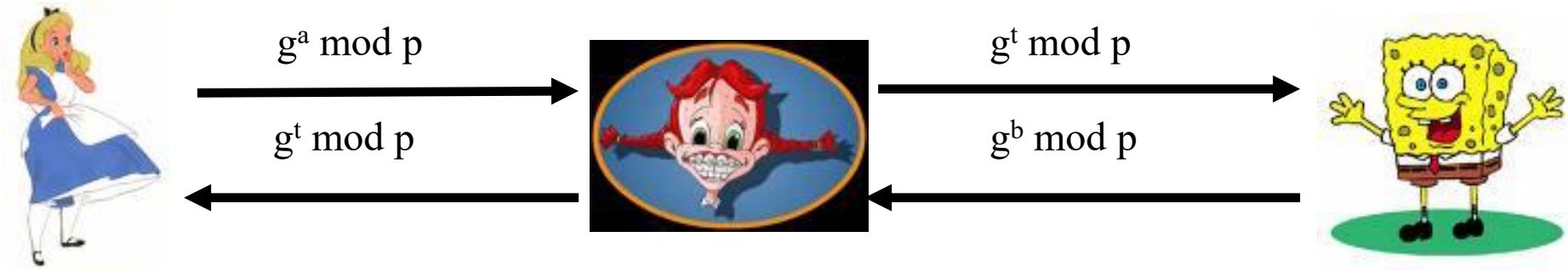
Diffie-Hellman

- Suppose that Bob and Alice use $g^{ab} \bmod p$ as a symmetric key
- Trudy can see
  - $g^a \bmod p$
  - $g^b \bmod p$

  Can she find $g^{ab} \bmod p$?
- Note $g^a g^b \bmod p = g^{a+b} \bmod p \neq g^{ab} \bmod p$
- If Trudy can find a or b, system is broken
- If Trudy can solve **discrete log** problem, then she can find a or b
- But, it's very very hard!!!

Diffie-Hellman Weakness

- Weak to man-in-the-middle (MITM) attack



generate a
$(g^t)^a \bmod p = g^{at} \bmod p$

generate t
$(g^a)^t \bmod p = g^{at} \bmod p$
$(g^b)^t \bmod p = g^{bt} \bmod p$

generate b
$(g^t)^b \bmod p = g^{bt} \bmod p$

- Trudy shares secret $g^{at} \bmod p$ with Alice
- Trudy shares secret $g^{bt} \bmod p$ with Bob
- Alice and Bob don't know Trudy exists!

Diffie-Hellman

- How to prevent MiM attack?
  - Encrypt DH exchange with symmetric key
  - Encrypt DH exchange with public key
  - Sign DH values with private key
  - Other?

  - Need some way for authentication

- You **MUST** be aware of MITM attack on Diffie-Hellman

Elliptic Curve Crypto (ECC)

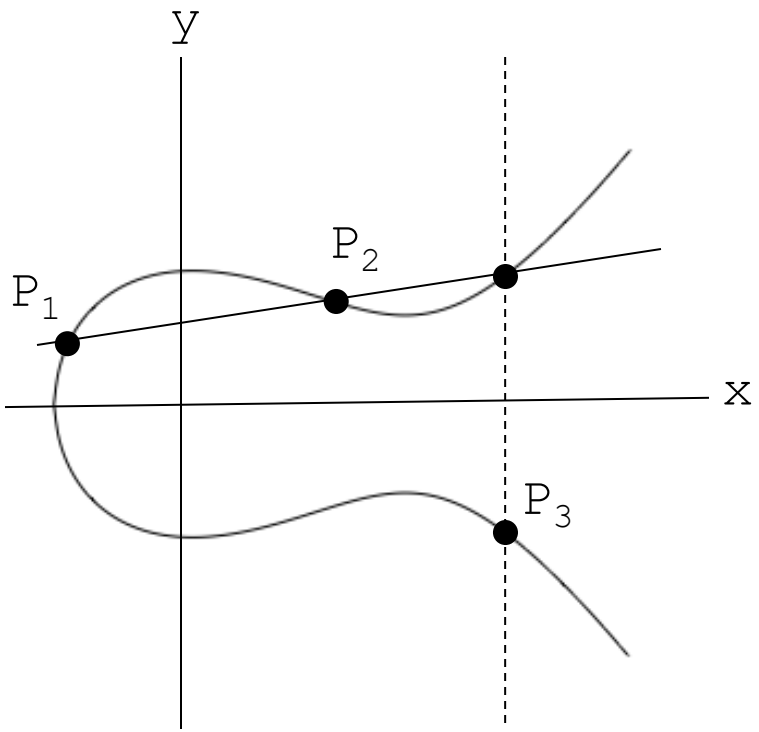
- Elliptic curve is **not** a cryptosystem
- Elliptic curves are a different way to do the math in public key system

- Elliptic curve versions of DH, RSA, etc.
- Elliptic curves may be more efficient
  - Fewer bits needed for same security
  - But the operations are more complex

What is an Elliptic Curve?

- An elliptic curve E is the graph of an equation of the form

$$y^2 = x^3 + ax + b$$



- Consider elliptic curve

$$E: y^2 = x^3 - x + 1$$

- If P_1 and P_2 are on E, we can define

$$P_3 = P_1 + P_2$$

as shown in picture

- Addition is all we need

Elliptic Curve Cryptography

- Use elliptic curves
  - Variables and coefficients are elements of a finite field
- 2 families
  - Prime curve over Z_p
  - Binary curve over GP(2^m)
- Elliptic curve over Z_p

$$y^2 \bmod p = (x^3 + ax + b) \bmod p$$

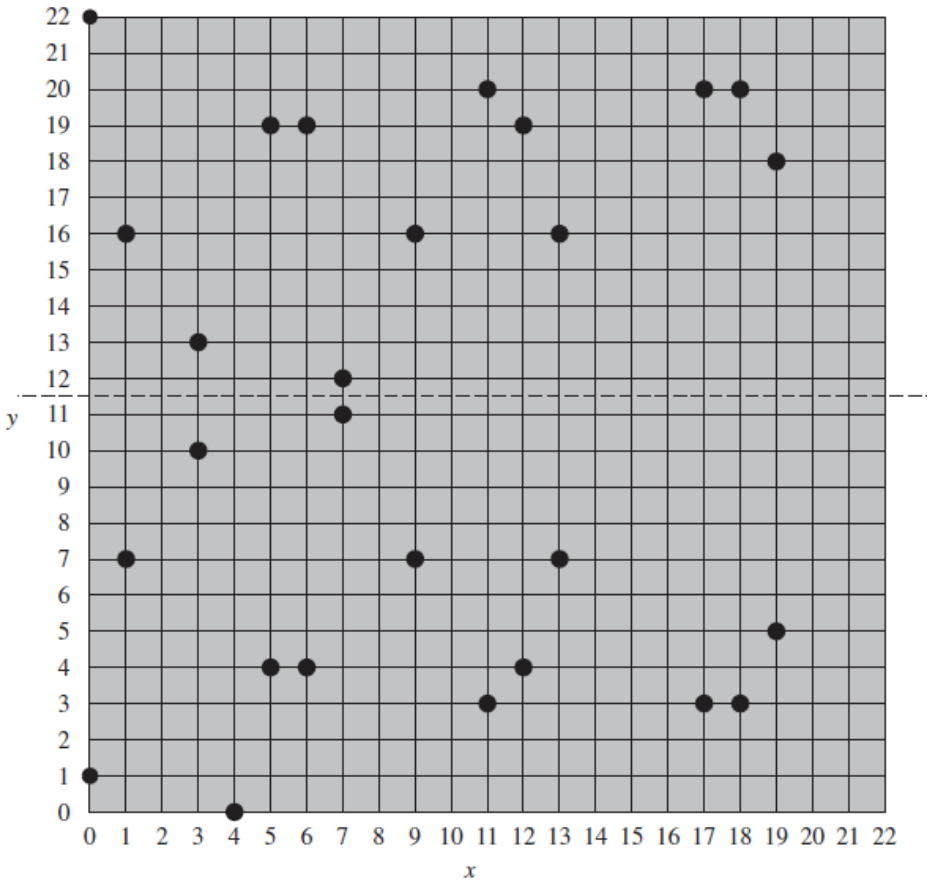
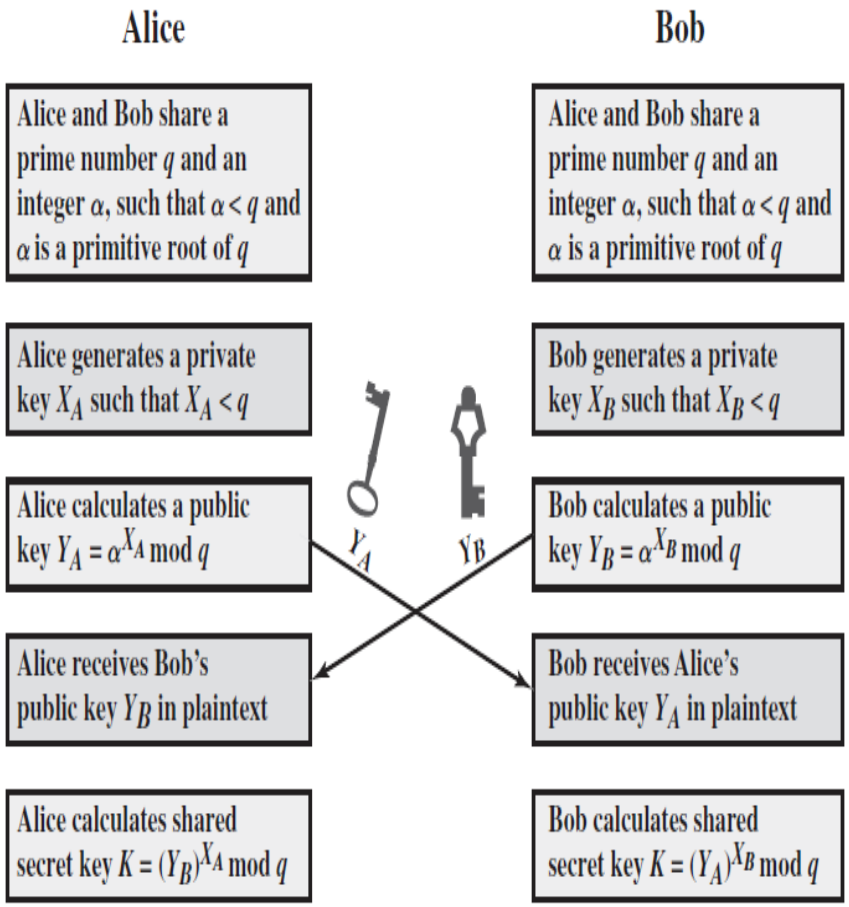


Figure 10.5 The Elliptic Curve $E_{23}(1, 1)$

DH vs. ECDH



Global Public Elements

$E_q(a, b)$	elliptic curve with parameters a, b, and q, where q is a prime or an integer of the form 2^m
G	point on elliptic curve whose order is large value n

User A Key Generation

Select private n_A	$n_A < n$
Calculate public P_A	$P_A = n_A \times G$

User B Key Generation

Select private n_B	$n_B < n$
Calculate public P_B	$P_B = n_B \times G$

Calculation of Secret Key by User A

$K = n_A \times P_B$

Calculation of Secret Key by User B

$K = n_B \times P_A$

Figure 10.7 ECC Diffie-Hellman Key Exchange

ECC Security

Table 10.3 Comparable Key Sizes in Terms of Computational Effort for Cryptanalysis (NIST SP-800-57)

Symmetric Key Algorithms	Diffie-Hellman, Digital Signature Algorithm	RSA (size of n in bits)	ECC (modulus size in bits)
80	$L = 1024$ $N = 160$	1024	160–223
112	$L = 2048$ $N = 224$	2048	224–255
128	$L = 3072$ $N = 256$	3072	256–383
192	$L = 7680$ $N = 384$	7680	384–511
256	$L = 15{,}360$ $N = 512$	15,360	512+

Note: L = size of public key, N = size of private key

Public Key Notation

- **Sign** message M with Alice's **private key:** $[M]_{Alice}$
- **Encrypt** message M with Alice's **public key:** $\{M\}_{Alice}$
- Then

$$\{[M]_{Alice}\}_{Alice} = M$$
$$[\{M\}_{Alice}]_{Alice} = M$$

Uses for Public Key Crypto

- Can do anything that can do with symmetric crypto
  - Confidentiality
    - Encrypt with public key
  - Integrity
    - Sign with private key

- Digital signature provides
  - Integrity
  - **Non-repudiation**

- No non-repudiation with symmetric keys

Non-repudiation

- Alice orders 100 shares of stock from Bob
- Alice computes **MAC** using symmetric key
- Stock drops, Alice claims she did not order

- Can Bob prove that Alice placed the order?
- **No!**
  - Since Bob also knows symmetric key, he could have forged message
- **Problem**
  - Bob knows Alice placed the order, but he can't prove it

Non-repudiation

- Alice orders 100 shares of stock from Bob
- Alice **signs** order with her private key
- Stock drops, Alice claims she did not order

- Can Bob prove that Alice placed the order?
- **Yes!**
  - Only someone with Alice's private key could have signed the order

- This assumes Alice's private key is not stolen (revocation problem)

Confidentiality and Non-repudiation

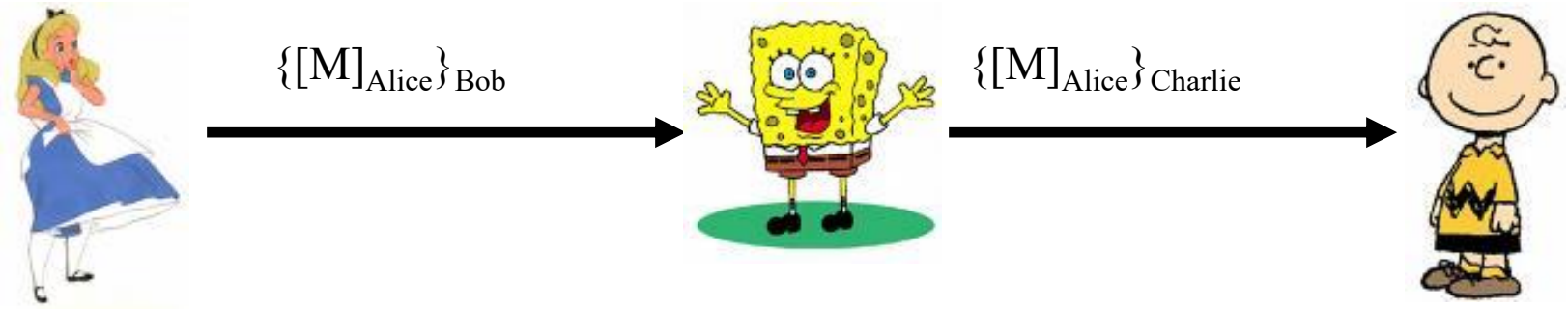
- Suppose that we want confidentiality and non-repudiation

- Can public key crypto achieve both?
- Alice sends message to Bob
  - **Sign and encrypt** $\{[M]_{Alice}\}_{Bob}$
  - **Encrypt and sign** $[\{M\}_{Bob}]_{Alice}$

- Can the order possibly matter?

Sign and Encrypt

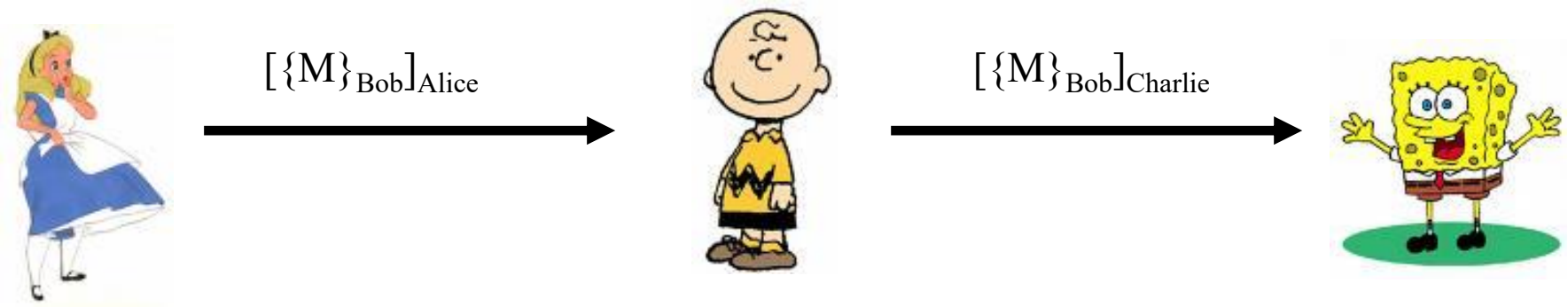
M = "I love you"



What is the problem?

Encrypt and Sign

M = "I found a great theory. It's …"



Charlie cannot decrypt M

What is the problem?

Symmetric Key vs Public Key

- Advantages of symmetric key crypto
  - **Speed**
  - No public key infrastructure (PKI) needed

- Advantages of public key crypto
  - **Signatures** (non-repudiation)
  - No shared secret

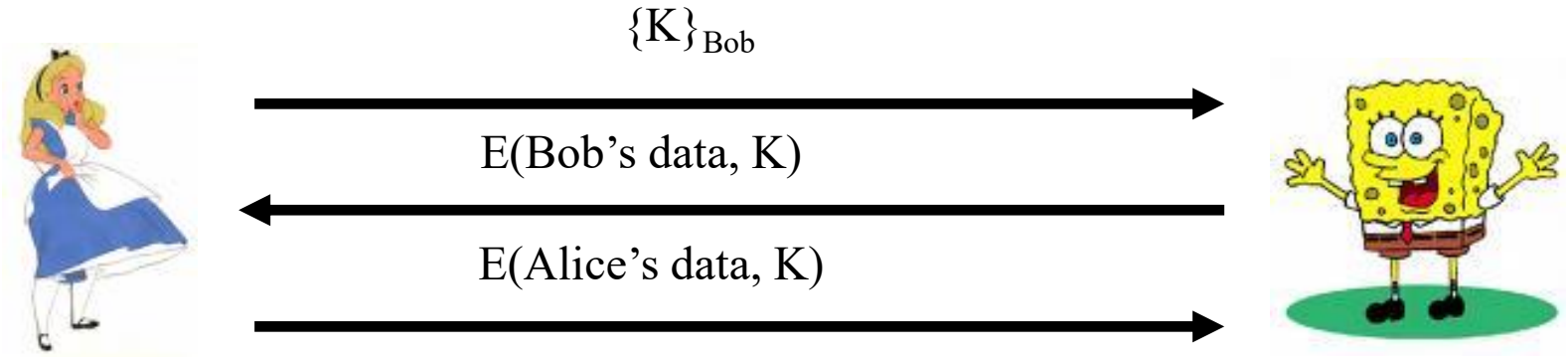
Notation Reminder

- Public key notation
  - Sign message M with Alice's **private key**
    - $[M]_{Alice}$
  - Encrypt message M with Alice's **public key**
    - $\{M\}_{Alice}$

- Symmetric key notation
  - Encrypt plaintext P with symmetric key K
    - $C = E(P,K)$
  - Decrypt ciphertext C with symmetric key K
    - $P = D(C,K)$

Real World Confidentiality

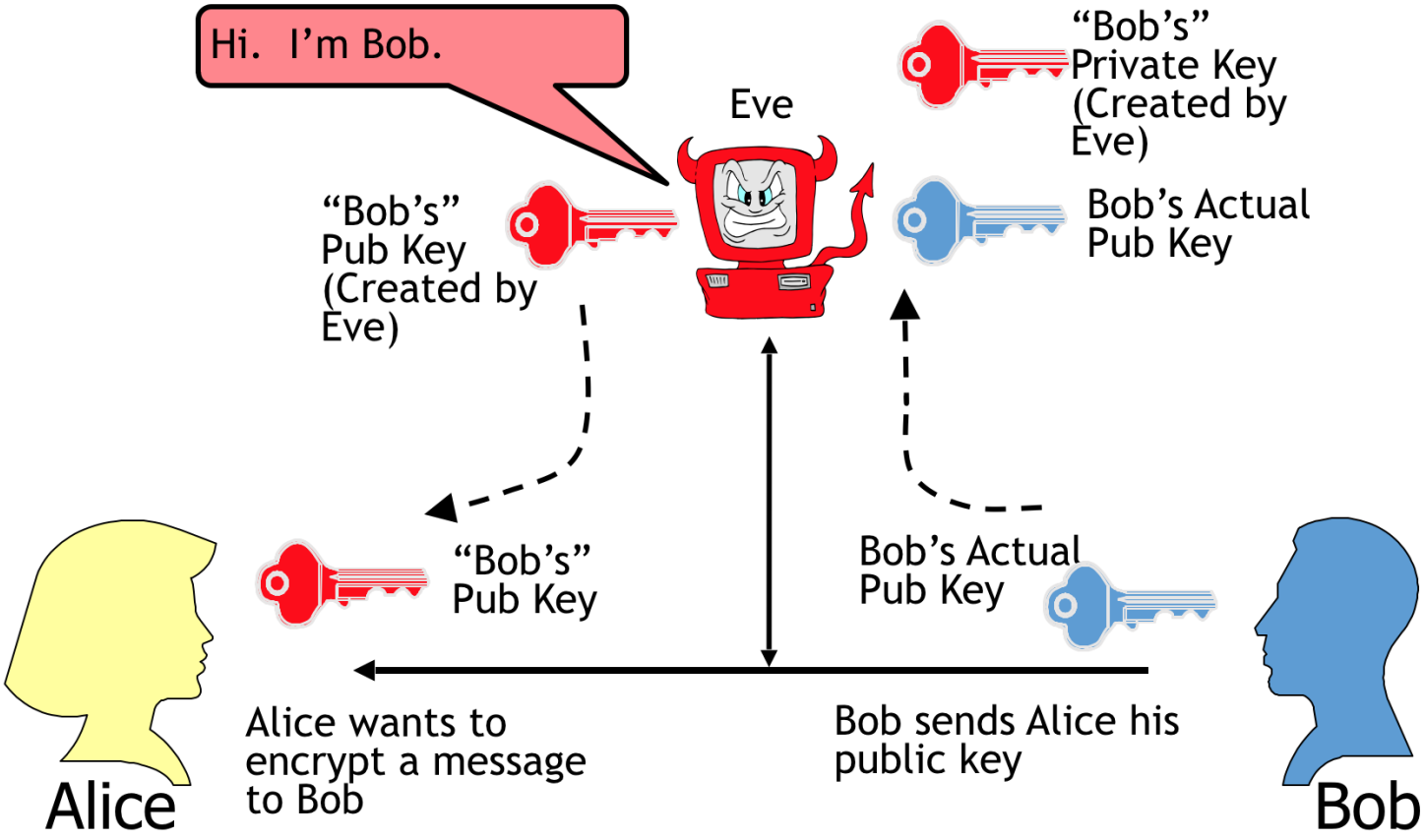
- Hybrid cryptosystem
  - Public key crypto to establish a key
  - Symmetric key crypto to encrypt data



Can Bob be sure he's talking to Alice?

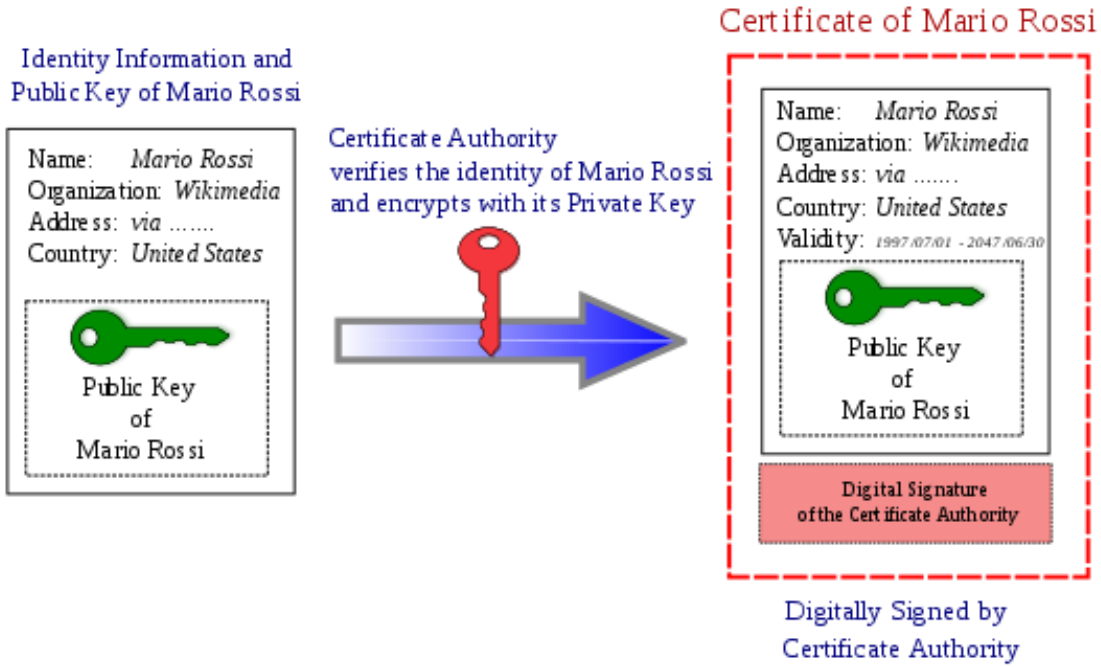
Public Key Certificate

- Public key crypto does not share secret key
- But, how can we get and verify Alice's public key?
  - Use certificates

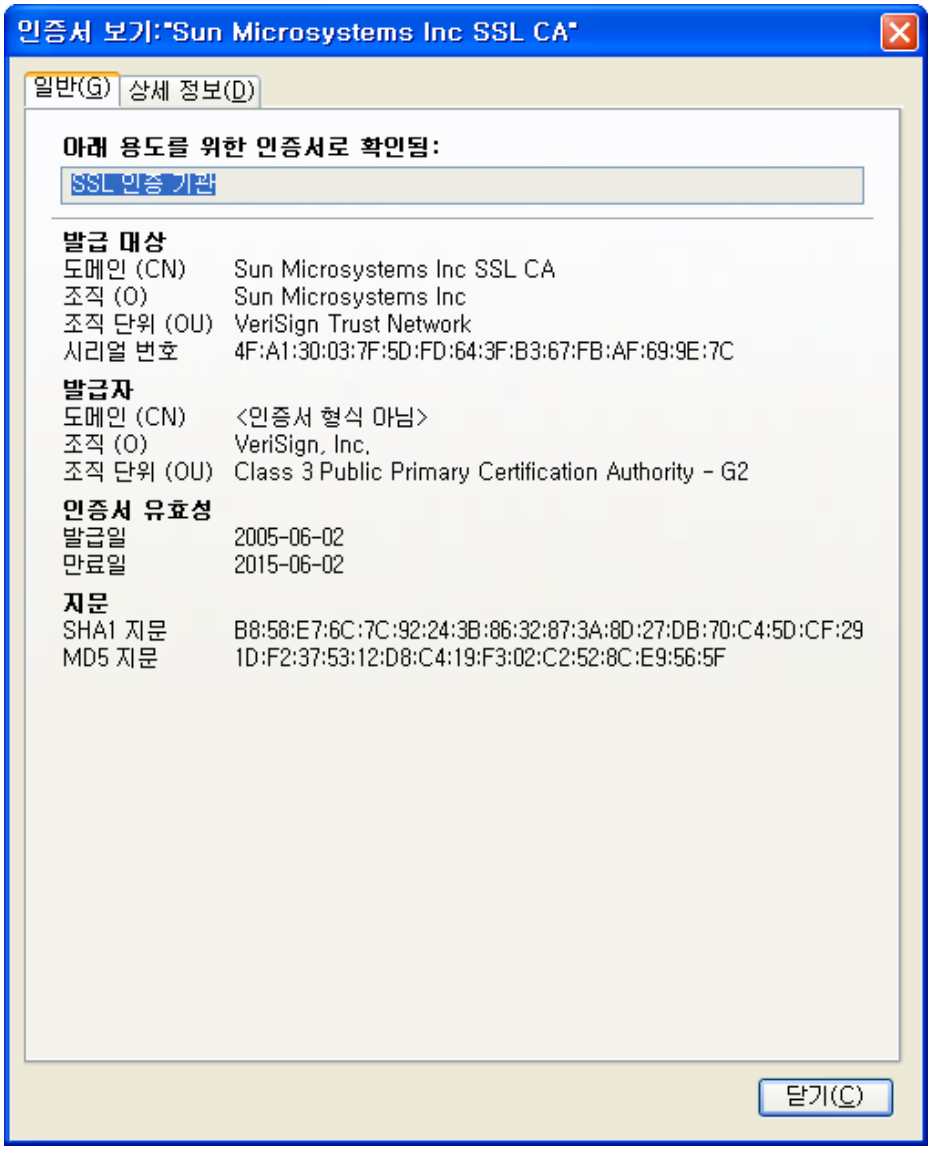


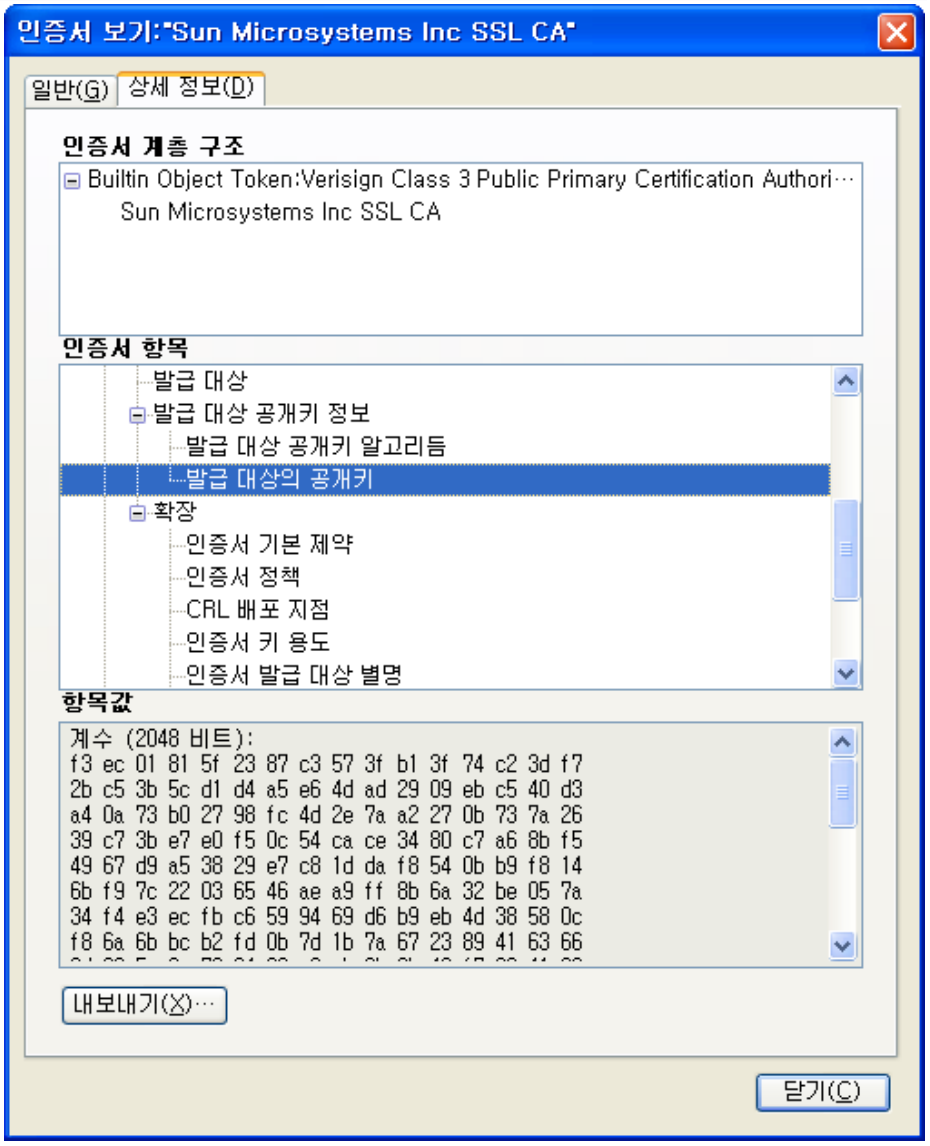
Public Key Certificate

- Digital certificate
  - Contains name of user and user's public key (and possibly other info)
  - Certificate is **signed** by the issuer (such as VeriSign) who vouches for it
  - Signature on certificate is verified using signer's public key



Public Key Certificate

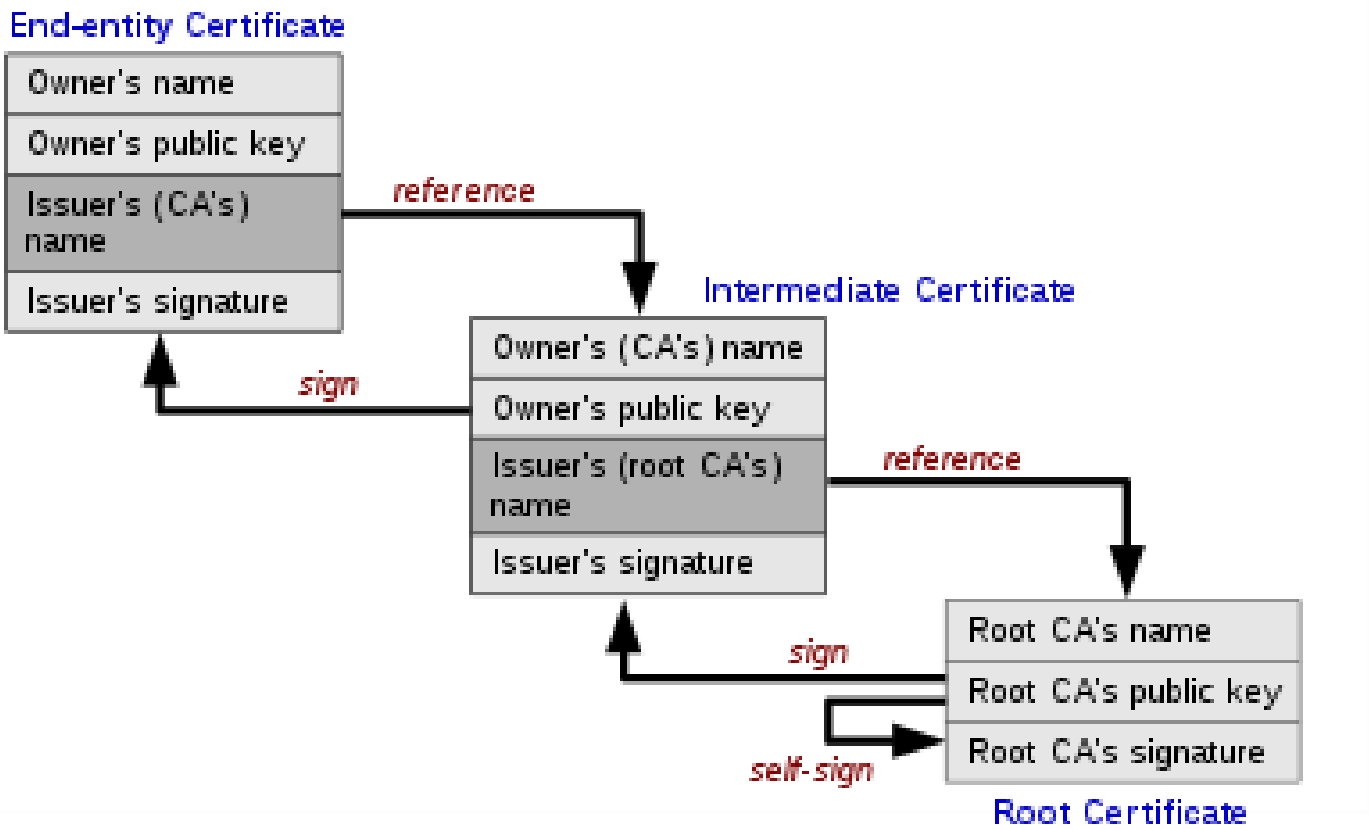




Certificate Authority

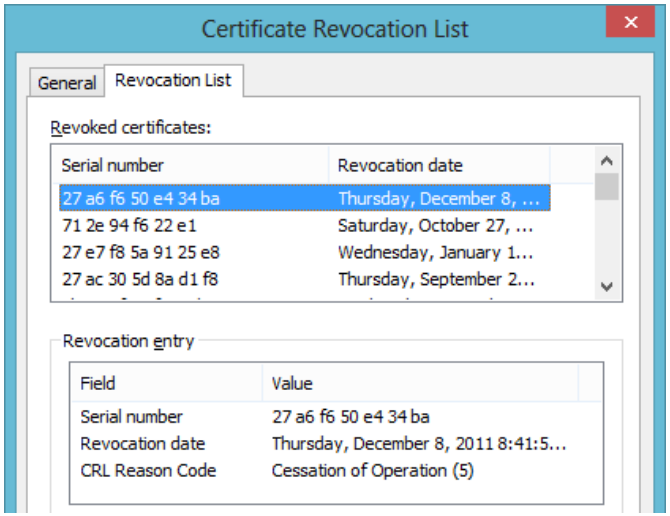
- Certificate authority (CA)
  - A trusted 3rd party (TTP) that issues and signs cert's
- Verifying signature
  - Verifies the identity of the owner of corresponding public/private key pair
  - Does **not** verify the identity of the source of certificate!
  - Certificates are public!
- Big problem if CA makes a mistake
  - a CA once issued Microsoft certificate to someone else!
- Common format for certificates is X.509

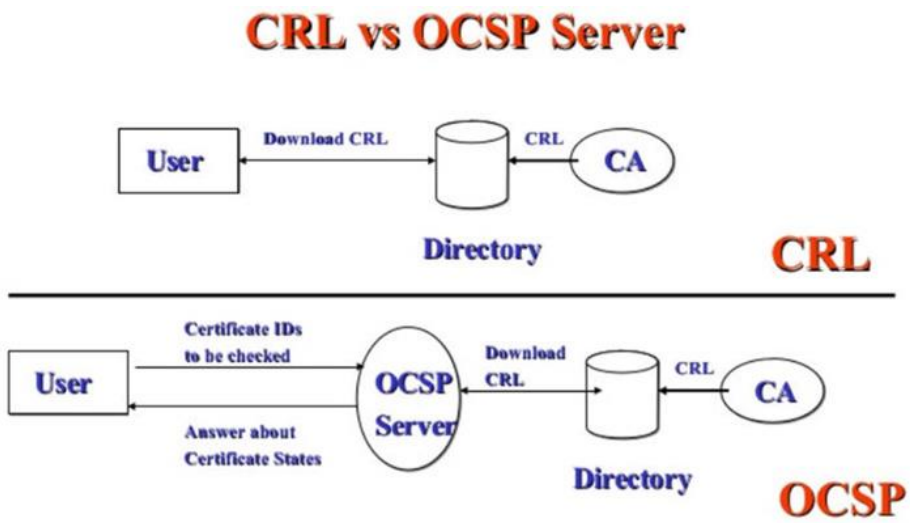
Certificate Authority



PKI

- Public Key Infrastructure (PKI)
  - Consists of all pieces needed to securely use public key cryptography
  - Key generation and management
  - Certificate authorities
  - Certificate revocation (CRLs), OCSP, etc.

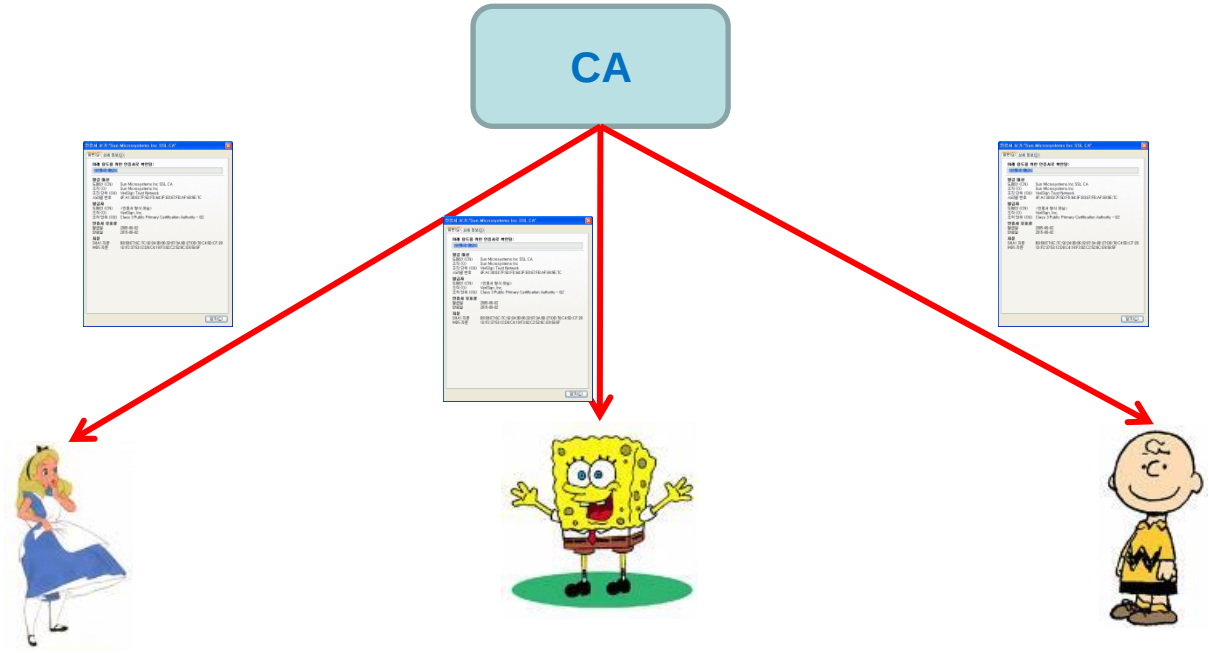




- No general standard for PKI
- We consider a few “trust models”

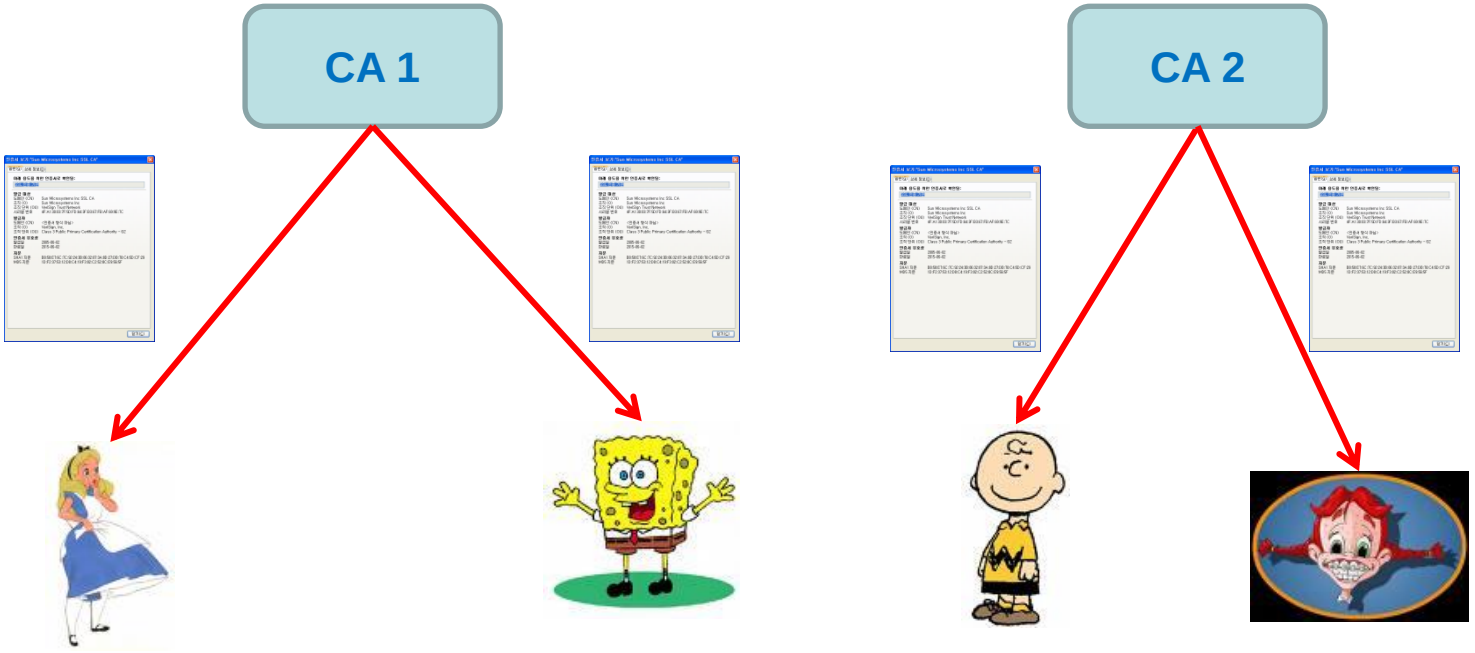
PKI Trust Models

- Monopoly model
  - One universally trusted organization is the CA for the known universe
  - Favored by VeriSign (for obvious reasons)
  - Big problems if CA is ever compromised
  - Big problem if you don't trust the CA!



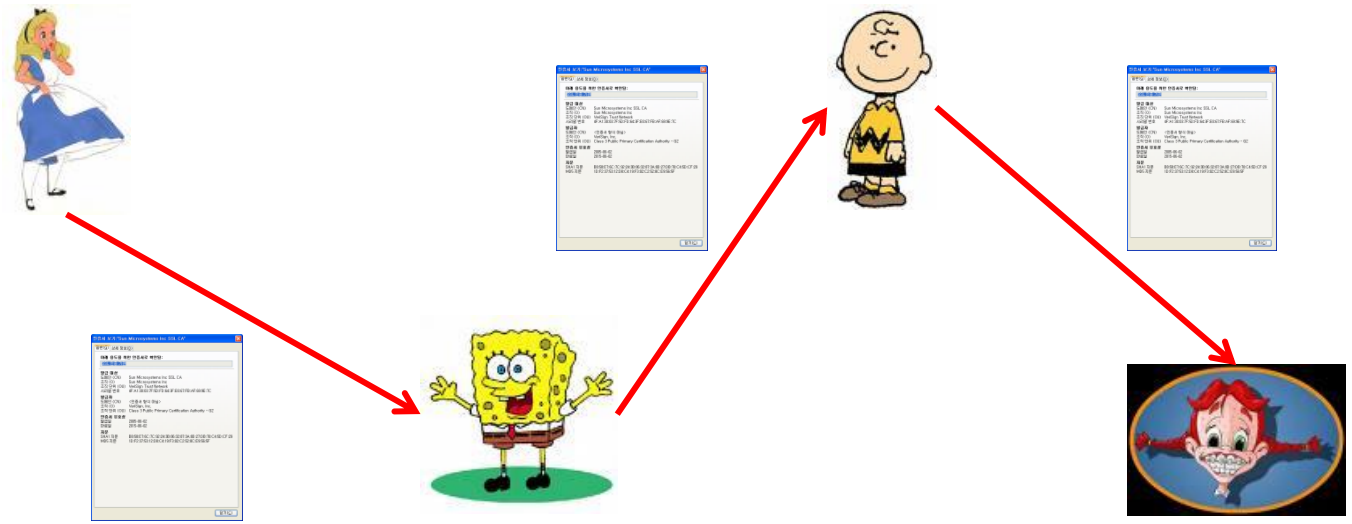
PKI Trust Models

- Oligarchy
  - Multiple trusted CAs
  - This approach used in browsers today
  - Browser may have 80 or more certificates, just to verify signatures!
  - User can decide which CAs to trust



PKI Trust Models

- Anarchy model
  - Everyone is a CA!
  - Users must decide which “CAs” to trust
  - This approach used in PGP (Web of trust)



Can Alice trust Trudy?